Pipedrive Sales Funnel Analysis: Walkthrough

This document details the process, architecture, and key decisions made in building a dbt project to analyze the Pipedrive sales funnel. It incorporates initial data exploration findings and explains the logic behind the final data models.

1. Project Setup & Data Ingestion

The project was set up following modern data engineering best practices:

- **Infrastructure**: A trial Snowflake account was created with a dedicated database (`pipedrive_analytics`) and distinct schemas for each stage of transformation (`raw`, `staging`, `intermediate`, `production`). This separation ensures clear data lineage and access control.
- **Tooling**: `dbt-snowflake`, and the `dbt-utils` package were installed.
- **Data Loading**: The raw CSV files were treated as seeds. In a production environment, this step would be handled by an orchestrator like Airflow, ingesting data into the `raw` schema. Data ownership is clearly defined: Data Engineers own the `raw` layer, and Analytics Engineers own the subsequent layers.
- **Configuration**: The `profiles.yml` and `dbt_project.yml` files were configured to connect to Snowflake and define the layered modelling structure.

2. Initial Data Exploration & Key Assumptions

Before modelling, a thorough data exploration was conducted. This surfaced several critical issues and led to key assumptions that shaped the project. The following points are crucial for any stakeholder to understand the context of the final report.

In-Scope Findings & Assumptions:

- **Duplicate `activity_id`s**: The `activity` source table contained 11 non-unique `activity_id`s (e.g., `206894`). This points to potential data corruption or sync anomalies.
    - **Decision**: Under the assumption that this is corrupted data, these records were **excluded** in the staging layer to ensure the integrity of downstream analysis.
- **Referential Integrity Issues**: There was a significant mismatch in `deal_id`s between the `activity` and `deal_changes` tables, with only 8 deals in common.
    - **Assumption**: This is due to the dataset being a small, sampled version of production data. The models are built assuming that in a real-world scenario, a deal with an activity would have a corresponding creation event.

- **PII in `users` Table**: The `users` table contains names and emails. To comply with data privacy regulations and best practices, this PII needs to be handled carefully.
    - **Decision**: The `user_name` and `email` columns were **hashed** in the staging layer to create anonymized identifiers. This removes the PII risk while still allowing for user-level analysis.
- **Inconsistent Naming**: Minor inconsistencies (e.g., "Qualified lead" vs. "Qualified Lead") were found between `stages.csv` and `fields.csv`.
    - **Decision**: While these can be cleaned with SQL functions, they represent a data governance issue. The model uses `LOWER()` for robust joining but highlights the need for source data alignment.
- **Activities and Their Relation to Funnel Steps:** All deal changes related to transition between different stage id's happen at the exact same timestamp.
    - If these timestamps were different, some analyses around movement of deals and their direction could also provide insights into the funnel as to where deals could be stalled / moving backwards.
    - Interestingly enough, there were some deals that did go to the same step (Lost) multiple times (twice).
- **Timestamps**: All timestamps are assumed to be in **UTC**. This is a best practice, allowing for consistent calculations. Local time conversions should happen at the final presentation layer (e.g., in a BI tool).
- **Macro:** A custom generate_schema_name macro has been added in to ensure that we can work across multiple schemas in production and also in development (when working locally, someone can just add their name to the dev_suffix variable in project.yml file). Things such as lost_reason and stage_name mapping could also been turned to macros if needed.

3. The Layered Data Model: A Step-by-Step Walkthrough

The dbt project is structured in three layers: Staging, Intermediate, and Marts (Production).

Step 0: Raw Layer (Raw data lands from data producers)

In this case, the raw layer is where the initial csv files have been materialised as seeds.

Step 1: Staging Layer (The Clean-up Crew)

The goal here is to clean and prepare the raw data. Staging models perform basic transformations such as renaming, typecasting, deduplication and case when's on a 1:1 basis with the source tables. They are materialized as views for performance, except for cases when we expect the data volume to be quite high and the source to be reused in several cases.

Freshness checks, unique, not null, accepted values & referential integrity checks are applied here along with some basic documentation.

Not all staging models were needed for further analysis in this case but they've been created nonetheless.

Activity & Deal Changes

With the assumption that these two models would become quite big in production, these have been modelled as incremental models.

We may have sync anomalies (same activity recorded multiple times but in different states) in the source table, or the data producer creating the data had some duplicated joins, or we just received corrupted data (~11/4.6k rows).

How to deal with these would be a business outcome (do we keep both, which one to keep or exclude both) and can be discussed with stakeholders - for now, I've excluded this data under the assumption that it is corrupted data.

Deal changes table has ~2k distinct deal ids, activity has ~4.6k but only 8 unique deal ids are present in both tables. The assumption for this discrepancy is that the current dataset is a sampled version of production and hence does not have this 1-1 mapping but ideally, this should not be the case. This creates an issue with funnel sub-step analysis because only one deal matches the criteria to go into Step 2.1: Sales Call 1and no deals go into Step 3.1: Sales Call 2 in the data given making these rather redundant but these have still been modelled in anyway

Step 2: Intermediate Layer (The Logic Engine)

This is where the core business logic is implemented. These models are reusable building blocks for various analyses. They are materialized as tables to speed up downstream queries.

`int_dim_date.sql`

- **Purpose**: To create a dim_date/dim_timestamp table that could be used to join to any further analyses if needed
- **Logic**:
    1. It uses a generic generator function to create ~20 years worth of dates and can run as a one-off materialised table.
    2. I did not use this further in my analysis as the requirement was monthly and I did not see any missing months but it could've been useful to create something at he level of granularity needed (hourly/daily) and then aggregate it upwards(monthly).

`int_deal_stage_history.sql`

- **Purpose**: To construct a complete, chronological history of every stage a deal has entered
- **Logic**:
    1. Filters stg_deal_changes for stage_id changes.

2. Creates a surrogate key (deal_stage_change_id) using dbt_utils.generate_surrogate_key.
    - Not particularly useful in our case as all deals have the same timestamp for chanes to stage_id but typically would be useful as a differentiator.
3. Computes valid_from and valid_to using lead() to capture stage duration.
    - Same as above, not particularly useful for us right now but combined with the above, it ensures every stage change is uniquely tracked, which is critical for non-linear funnel analysis.
4. Joins to stg_stages to include stage names.
5. Incremental logic: Only new stage changes since the latest valid_from are processed.
    - Assuming append only changes happen otherwise we can switch to merge strategy with deal_stage_change_id as the unique key with some days of lookback.

`int_deals_with_kpis.sql`

- **Purpose**: To enrich the deal stage history with our defined KPI names (funnel steps and sub-steps).
- **Logic**:
    1. Pre-aggregation: Scans the activity table once to flag completed “Sales Call 1” and “Sales Call 2” per deal.
        - If the number of sub-steps begins to grow, it would make sense to
    2. Efficient Joins: LEFT JOIN to stage history instead of row-by-row correlated queries.
    3. CASE statements: Map stage names to funnel_step and, using activity flags, identify funnel_sub_step.

Step 3: Production Layer (Marts, Ready for Reporting)

This is the final layer, intended for consumption by BI tools or business users. The data is aggregated and presented in a clean, wide format.

`rep_sales_funnel_monthly.sql`

- **Purpose**: Create a monthly sales funnel report for BI consumption.
- **Logic**:
    1. Pulls enriched data from int_deals_with_kpis.
    2. Unpivoting: Funnel steps (funnel_step and funnel_sub_step) are flattened into a single kpi_name column.
        - This is to ensure that we can also provide reporting on sub-steps but as mentioned, with the data that we have, we only get one deal for Sales Call 1 and none for Sales Call 2
    3. Aggregation: Groups by month, kpi_name, and funnel_step to count distinct deal_stage_change_id.

        - Since the aggregation level is quite high (monthly), the model has been kept as a table to ensure accurate results at all times.
        - Still kept as table vs view to ensure that we don’t run into any performance issues when some BI tool is connected to it.
    4. Column deals_entered_stage_count represents how many deals entered each stage/KPI in a given month.

This robust, layered, and optimized approach results in a final report that is not only accurate but also resilient to common data quality issues and reflective of the complex, real-world behavior of a sales funnel.